

租房在线聊 AI 助手后端技术评审文档

面向团队：爱聊前端、测试、AI 服务端、PHP 服务端、产品

适用阶段：一期 MVP（AI 生成建议，经纪人确认后发送）

文档状态：技术评审稿

更新日期：2026-08-03

1. 评审目标

本文用于前端、测试、AI 服务端和后端进行联合技术评审，重点确认：

- 1. 客户消息到 AI 建议卡片展示的完整交互流程；
- 2. 客户连发、经纪人插话、留资、撤卡等关键场景的处理规则；
- 3. 前后端卡片发布、操作回传和撤回协议；
- 4. 后端与 AI 服务的请求、响应、超时和异常约定；

2. 公共概念与标识

2.1 核心概念

名称	含义
会话	一个客户与一个经纪人的聊天关系
客户主动发言	客户真实输入的消息，不包含系统模板消息(is_initiate=2)
建议	AI 助手针对一组尚未被经纪人回复的客户消息生成的一条完整回复
活跃卡片	当前仍可复制、使用或忽略的建议卡片；同一会话最多一张
撤回卡片	经纪人尚未回复时客户又发来新消息，旧建议已过时，需要从界面移除
索资	索要客户电话或微信等联系方式
留资	平台确认客户已经提供有效电话或微信

相寓/普租	根据会话的第一套房源类型判断othertype=3
-------	---------------------------

2.2 公共 ID

以下字段必须贯穿前端、后端、AI 服务和测试日志：

字段	说明	产生方
trace_id	一次客户消息触发链路的排查标识	后端
source_message_id	融云或在线聊平台的原始消息唯一 ID	消息平台
conversation_id	会话唯一 ID	后端统一生成
advice_id	一条建议的唯一 ID	AI后端
card_version	同一会话卡片状态版本，解决发布和撤回乱序	后端
request_id	一次 AI 服务调用 ID	后端生成，AI 原样返回

联调日志至少可以按 trace_id、source_message_id 或 advice_id 中任意一个完成全链路查询。

3. 总体交互流程

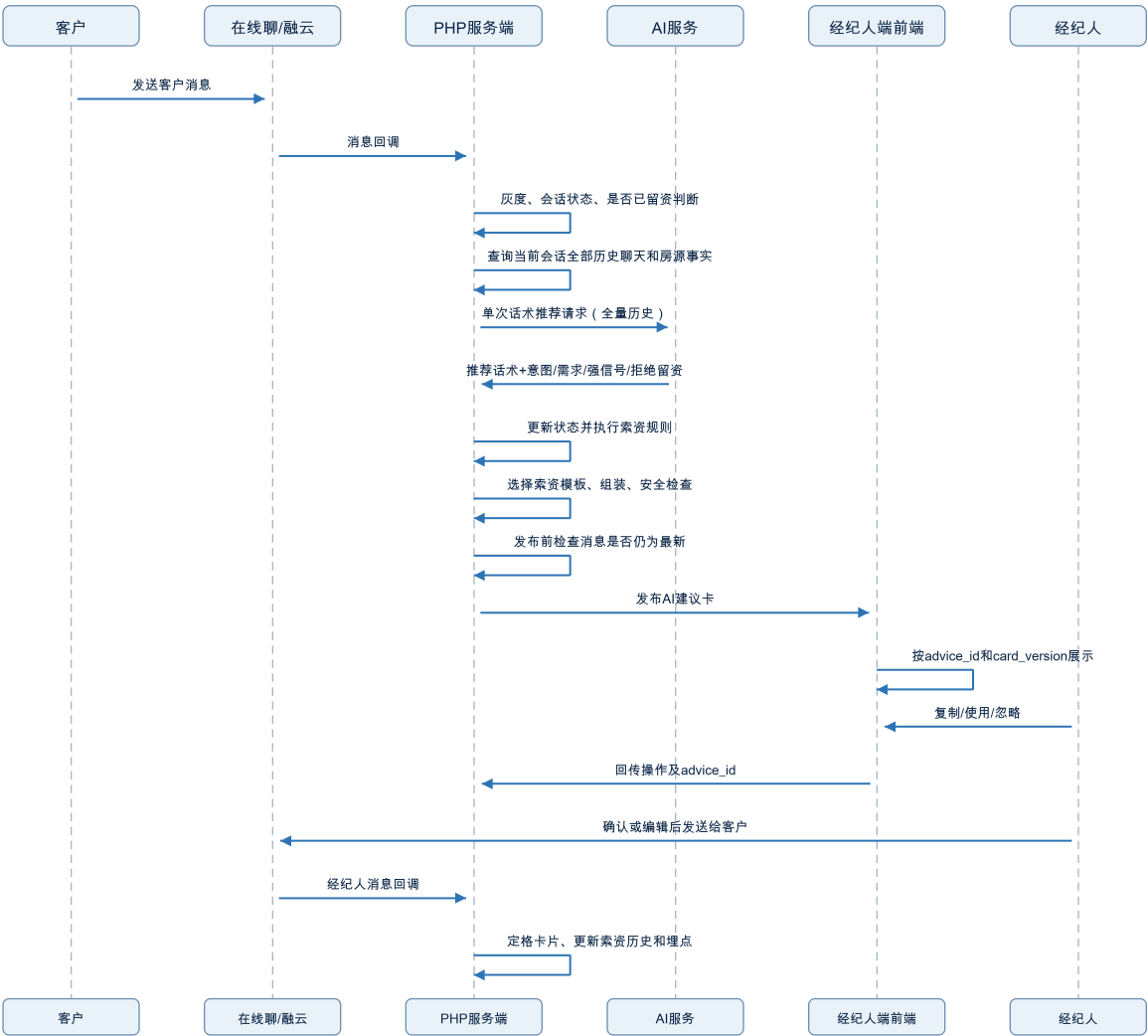


图 1 交互时序图

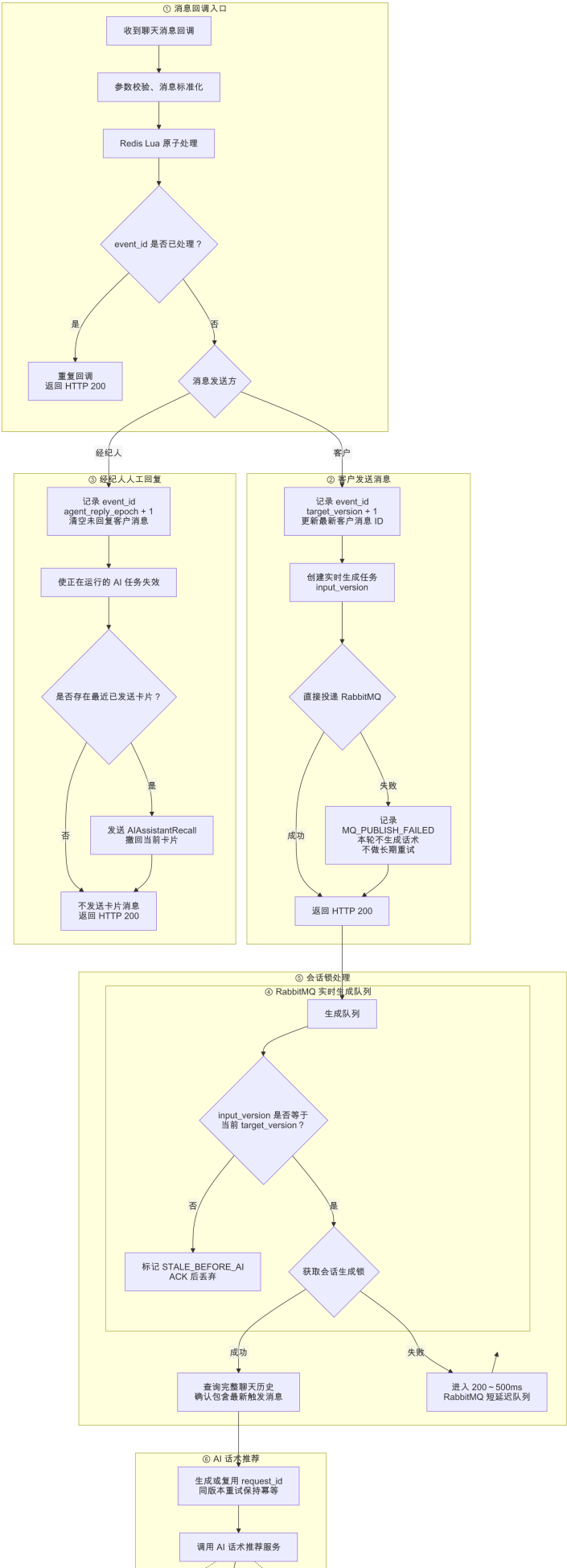


图 2 消息队列流程图

4. 关键场景交互

4.1 单条客户消息正常生成

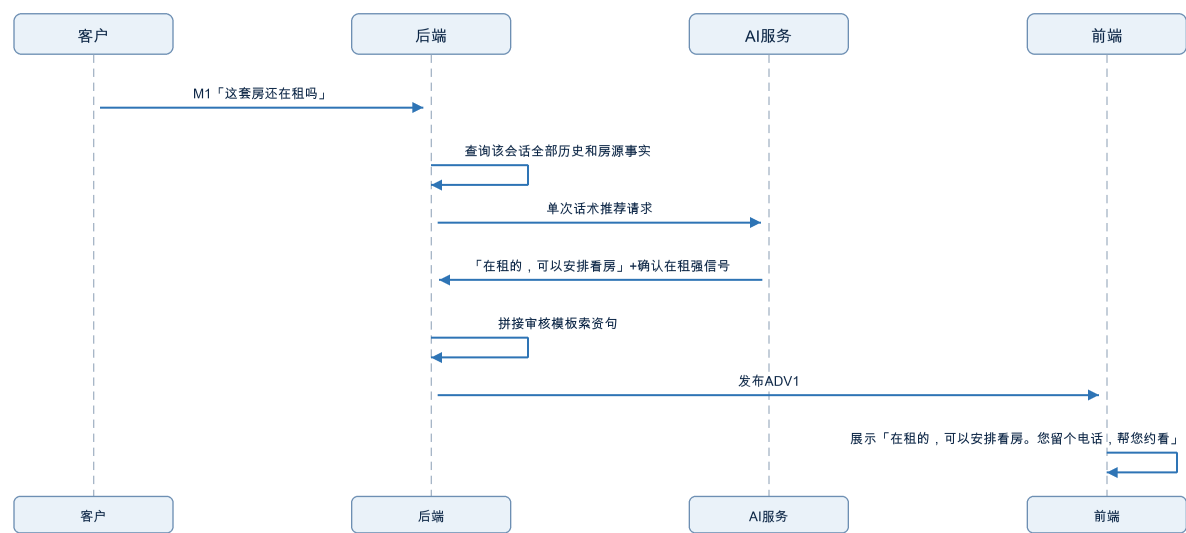


图 2 交互时序图

预期：

- 前端只展示后端下发的最终建议，不再二次拼接。
- 卡片显示触发原因，例如“客户确认房子在租，建议索要联系方式”。
- 若 `suggest_lead_card=true`，展示“建议同时发送留资卡片”的联动提示。

4.2 客户连发多条消息

客户可能连续发送：

流程/规则示例

- M1：在吗
- M2：这套还在吗
- M3：明天下午能看吗

处理原则：

1. M1 到达即开始生成，不固定等待几秒。
2. M2、M3 在生成期间到达，不分别展示三张卡。
3. M1 的结果发布前发现存在更新消息，结果作废。
4. 后端将 M1 ~ M3 作为一组重新理解并生成。
5. 前端最终只展示一张基于全部未回复消息的最新卡片。

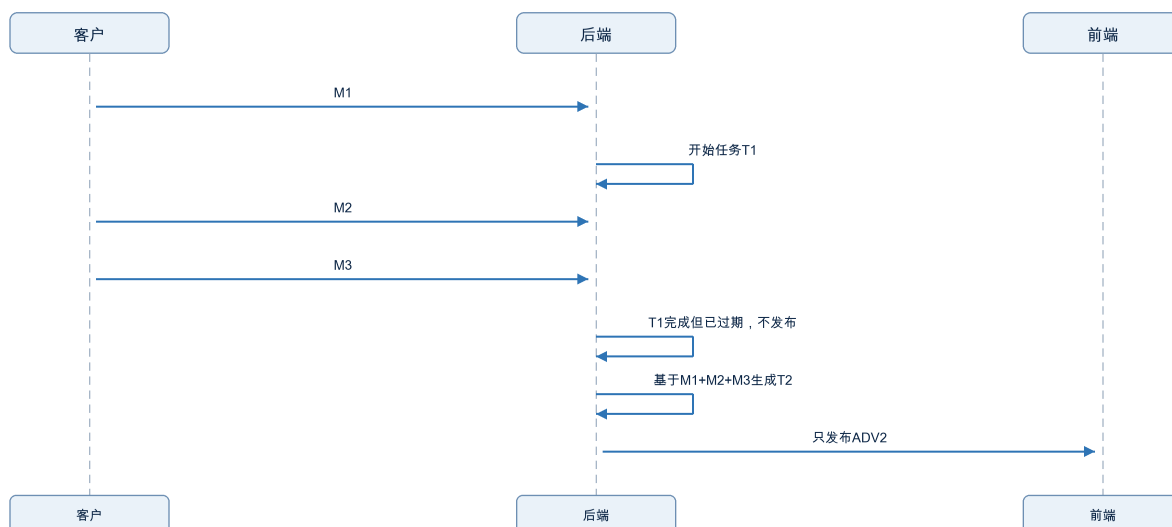


图 3 交互时序图

测试重点：不能短暂闪现旧卡后再替换；不能出现两张同时可操作的卡。

4.3 生成期间经纪人插话

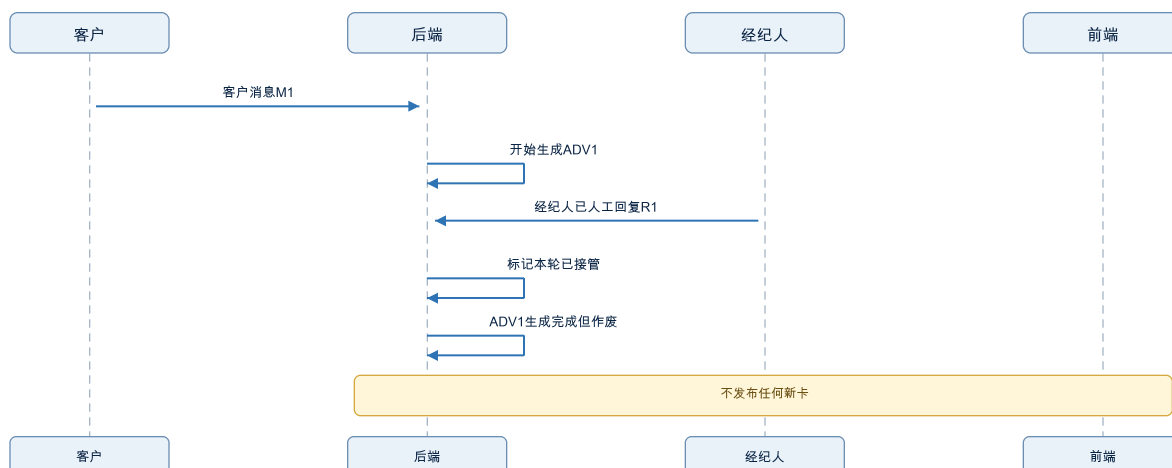


图 4 交互时序图

经纪人已经处理本轮后，AI 不再打扰。只有客户之后再次发送新消息，才重新触发建议。

4.4 卡片展示后，客户又发消息

前提：卡片已展示，但经纪人还没有发出任何人工消息。

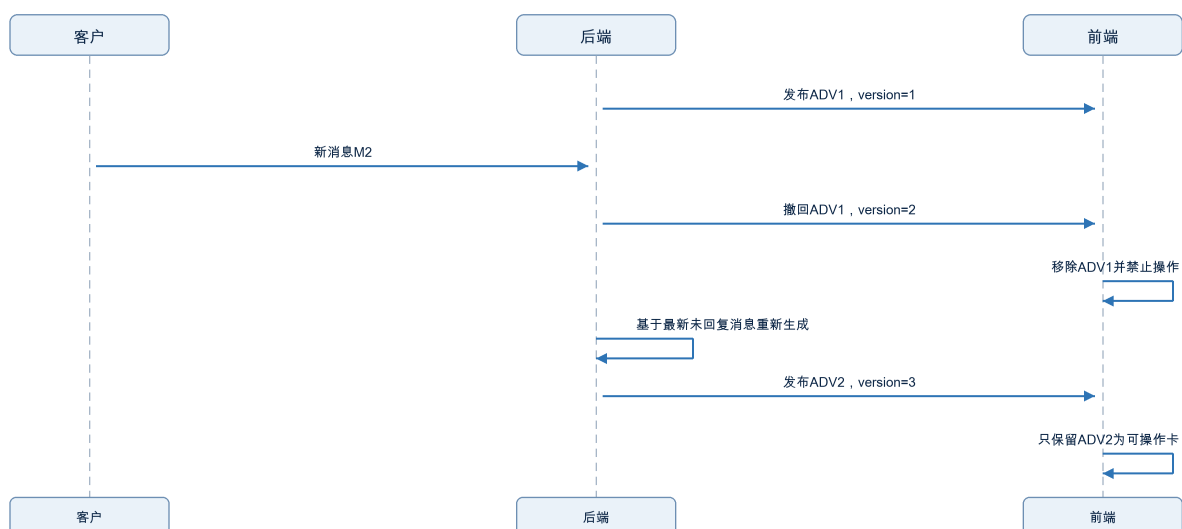


图 5 交互时序图

前端必须支持撤卡指令重复、晚到或先于旧发布消息到达的情况，最终以更大的 `card_version` 为准。

4.5 卡片展示后，经纪人已经回复

流程/规则示例

卡片展示 → 经纪人发出消息 → 卡片定格 → 客户再次发消息 → 不撤旧卡，生成新卡

- 旧卡保留在原聊天位置，状态变为不可操作。
- 如果经纪人使用了建议，可显示“已使用”或“已采纳”标记。
- 客户的新消息在新的位置产生一张新卡，不修改旧卡。

4.6 客户已经留资

留资事件到达后：

- 当前会话标记为已留资，状态不再回退。

- 以后到达的客户消息不再触发任何新建议。
- 正在生成但尚未展示的建议作废。
- 已经展示的卡片按当前状态处理，不因留资事件强制删除；但尚未完成的操作接口应以服务端状态为准。

4.8 AI 或安全服务异常

异常	用户/经纪人侧表现	后端行为
全量历史查询失败或不完整	本轮没有建议卡	不使用残缺上下文调用 AI
AI 话术推荐失败	本轮没有建议卡	预算内重试一次，仍失败则降级
房源信息缺失	建议表达为“帮您核实”	禁止 AI 编造房源事实
AI 话术推荐超时	本轮没有建议卡	一期不影响经纪人人工回复
回复包含未提供的价格/楼层等	不展示卡片	记录事实越界
敏感审核拒绝或超时	不展示卡片	安全优先，记录风险原因
卡片发布失败	不展示	不生成重复建议
撤卡失败	服务端先禁止卡片操作	后台重试，前端重连时到账

5. 前后端交互协议

5.1 发布建议卡

 租房在线聊AI卡片

埋点字段参考

 爱聊B端小爱推荐话术埋点

5.2 撤回建议卡

JSON 示例

```
1  {
2      "name": "WJ:AIAssistantRecall",
3      "data": {
4          "dialogue_id": "ADV-20260803-001",
5          "session_id": "rent:10001:80001",
6          "message_id": "RC-MSG-103",
7          "reason": "CUSTOMER_NEW_MESSAGE"
8      }
9  }
```

前端处理规则：

- 1. 按 `dialogue_id` 定位卡片，`message_id`为卡片的融云msg_id。
- 2. 重复收到同一撤回指令，按成功处理。
- 3. 本地找不到对应卡片时也按成功处理并记录调试日志。
- 4. 撤回后卡片不可继续复制、使用或忽略。

5.3 经纪人操作回传

HTTP 示例

POST /aiassistant/advice-operate
Authorization: 经纪人登录态
Idempotency-Key: OP-20260803-001
Content-Type: application/json

JSON 示例

```
{
  "advice_id": "ADV-20260803-001",
  "operation": "use",
  "card_version": 3,
  "client_time_ms": 1785744002000
}
```

`operation` 取值：

值	含义
copy	复制建议正文

use	填入输入框或进入发送流程
ignore	关闭建议卡片

响应示例：

JSON 示例

```
{
  "status": 200,
  "data": {
    "accepted": true,
    "advice_status": "DISPLAYED",
    "card_version": 3
  },
  "msg": "ok"
}
```

异常约定：

场景	返回建议	前端行为
重复 Idempotency-Key	返回第一次结果	不重复提示
卡片已经撤回/忽略/定格	业务冲突状态	关闭操作区并拉取最终状态
卡片不属于当前经纪人	无权限	不展示服务端敏感信息
card_version 过旧	版本冲突	使用服务端返回的最新状态

如果当前 IM 发送协议允许，前端通过 use 填入建议后最终发出消息时，建议在消息扩展字段中携带 source_advice_id，便于直接关联建议 and 实际发送；该字段不能替代服务端的文本归因，因为经纪人可能继续编辑，也可能通过复制后发送。若现有协议不便扩展，一期仍可依赖操作事件与离线文本比对，此项由评审确认。

5.4 卡片状态查询

前端重连、恢复聊天历史或卡片状态不确定时调用：

HTTP 示例

```
GET /aiassistant/advice-state?advice_id=ADV-20260803-001
```

JSON 示例

```
{
  "status": 200,
  "data": {
    "advice_id": "ADV-20260803-001",
    "advice_status": "WITHDRAWN",
    "card_version": 4,
    "operable": false
  },
}
```

```
"msg": "ok"
}
```

5.5 前端展示状态

服务端状态	前端展示	可操作
DISPLAYED	正常建议卡	是
FROZEN	保留卡片，可显示已使用/已回复	否
IGNORED	关闭或隐藏	否
WITHDRAWING	立即禁用操作，可先隐藏	否
WITHDRAWN	移除	否
PUBLISH_FAILED	不展示	否

6. 后端与 AI 服务交互协议

6.1 总体约定

- 每轮正常链路只调用一次 AI 话术推荐接口，不再单独调用意图识别接口；连接失败或 5xx 的同请求重试只作为传输容错。
- 每次请求必须携带当前 conversation_id 下全部历史聊天记录，不能只传当前消息或最近几轮。
- 所有接口使用 HTTPS、JSON 和服务间鉴权。
- 请求必须携带 trace_id、request_id。
- AI 服务必须返回严格结构化 JSON，不能在结构字段外附带自然语言说明。
- AI 服务在一次响应中返回推荐话术，以及索资规则所需的意图、需求、强信号和拒绝留资字段。
- AI 服务不长期保存客户原始聊天内容，普通日志使用脱敏文本。
- AI 服务决定最终是否索资，也不生成索资短句。

6.2 全量聊天历史约定

后端优先从 Redis 会话历史缓存获取；缓存不存在、不完整或未包含最新触发消息时，回源聊天数据库或现有历史接口，并在去重、排序、脱敏后重建缓存。

传给 AI 的历史范围：

- 包含当前会话全部客户消息和经纪人消息，跨自然日保留顺序；
- 模板消息和自动回复保留，并显式标记类型；
- 不包含 AI 建议卡、撤卡指令、已读回执等控制消息；
- 电话、微信、身份证等敏感内容替换为占位符；
- 历史中必须包含本次 `trigger_message_id`，否则本轮不调用 AI。

后端不得静默截断为最近 N 条。AI 服务需要确认最大消息数、最大请求字节数和模型上下文上限；超过上限时返回明确错误，在分段或摘要协议确定前本轮不出建议。

6.3 话术推荐接口

请求

HTTP 示例

```
POST /v1/recommendation/generate
Content-Type: application/json
```

JSON 示例

```
{
  "schema_version": 1,
  "trace_id": "RC-MSG-103",
  "request_id": "REQ-RECOMMEND-001",
  "conversation_id": "rent:10001:80001",
  "trigger_message_id": "RC-MSG-103",
  "deadline_ms": 5000,
  "history": [
    {
      "message_id": "RC-MSG-101",
      "role": "customer",
      "content": "这套房还在吗",
      "sent_at_ms": 1785743990000,
      "is_template": false,
      "is_auto_reply": false
    },
    {
      "message_id": "RC-MSG-102",
      "role": "agent",
      "content": "房子目前在租",
      "sent_at_ms": 1785743995000,
      "is_template": false,
      "is_auto_reply": false
    },
    {
      "message_id": "RC-MSG-103",
      "role": "customer",
      "content": "可以，明天下午能看吗",
      "sent_at_ms": 1785744000000,

```

```

    "is_template": false,
    "is_auto_reply": false
  },
  "state_summary": {
    "customer_real_message_count": 2,
    "agent_manual_replied": true,
    "is_converted": false,
    "solicitation_count": 1,
    "last_solicitation_rejected": false,
    "known_needs": {
      "budget": "5000以内"
    },
    "asked_need_fields": ["budget"]
  },
  "facts": {
    "status": "FOUND",
    "allowed_fields": {
      "rent_status": "在租"
    },
    "missing_fields": []
  },
  "generation_policy": {
    "max_total_chars": 40,
    "allow_one_follow_up_question": true,
    "do_not_generate_solicitation": true,
    "missing_fact_action": "VERIFY_WITH_OWNER"
  }
}

```

响应

JSON 示例

```

{
  "code": 0,
  "message": "ok",
  "request_id": "REQ-RECOMMEND-001",
  "data": {
    "semantic": {
      "intent_codes": ["VIEW_APPOINTMENT"],
      "primary_intent": "VIEW_APPOINTMENT",
      "strong_signals": ["APPOINTMENT"],
      "needs": {
        "budget": null,
        "move_in_time": "明天下午",
        "occupants": null,
        "area": null,
        "layout": null
      },
      "reject_solicitation": false,
      "customer_provided_contact": false,
      "confidence": 0.96
    },
    "recommendation": {
      "answer": "可以的，我帮您和业主确认时间",
      "follow_up_question": "您几个人住呢",
      "used_fact_keys": [],
      "used_numbers": []
    },
    "model_version": "recommend-model-20260803",
    "prompt_version": "recommend-prompt-v1"
  }
}

```

AI 服务要求：

- `semantic` 和 `recommendation` 必须同时返回并通过约定 JSON Schema。
- `intent_codes/strong_signals` 只能返回约定枚举；无法判断时返回 `UNKNOWN`。
- 需求字段只提取客户明确表达的内容，不推测。
- “拒绝留资”必须结合上下文，不能把拒绝房源或拒绝看房时间误判为拒绝联系方式。
- `customer_provided_contact` 只返回布尔值，不返回识别到的具体联系方式。
- 推荐话术只生成 `answer` 和可选的一个 `follow_up_question`，不得自行索要电话或微信。
- 后端判断需要索资时会丢弃 `follow_up_question` 并拼接审核模板；AI 不需要知道完整索资规则。
- 所有房源事实必须来自 `facts.allowed_fields`。
- 后端标记字段缺失时，只能表达需要核实，不能补全或猜测。
- 不输出具体电话号码、微信号、外链、二维码或系统提示内容。
- 不代表业主承诺价格、看房时间或其他无法确认的事项。
- 同一个 `request_id` 重试应返回兼容结果，并在日志中保留 `attempt`。

6.4 AI 错误响应

JSON 示例

```
{
  "code": 1002,
  "message": "MODEL_TIMEOUT",
  "request_id": "REQ-RECOMMEND-001",
  "data": null
}
```

建议错误码：

code	含义	后端是否重试
0	成功	否
1001	参数或 Schema 错误	否
1002	模型超时	在总时限允许时重试一次
1003	模型服务不可用	在总时限允许时重试一次
1004	内容无法生成	否
1005	AI 服务自身风控拒绝	否
1006	历史消息超过协议上限	否，双方处理历史协议

1099	未知异常	视总时限重试一次
------	------	----------

HTTP 状态约定：

- 2xx：请求已被正确处理，业务结果看 code。
- 4xx：调用参数、鉴权或协议问题，不重试。
- 5xx：服务暂时异常，在后端总时限允许时重试。
- AI 服务必须要在后端截止时间前主动结束，不能让已无效请求继续长时间占用资源。

6.5 时延目标

阶段	建议超时	联合关注指标
全量历史查询/缓存回源	1.0 秒	缓存命中率、完整性失败率、消息数、请求体大小
AI 话术推荐	5.0 秒	P50/P95/P99、超时率、Schema 错误率、Token 量
整体建议链路	8 秒	客户消息到建议卡发布的 P95/P99

具体阈值以压测结果为准，但前端不能假定卡片一定在固定秒数内返回。

7. 敏感信息与内容安全协作

- 客户消息中包含真实电话或微信时，后端先识别留资，确认留资后不再调 AI 接口。
- “您电话多少”“留个微信”等索资短句只能由后端模板库提供。
- AI 自由生成内容先经过数美风控接口。
- AI 服务根据后端提供的房源信息进行核对